
Embedding-Augmented Hybrid Retrieval with Quantized Semantic Indexing

Pratham Jain

Motilal Nehru National Institute of Technology Allahabad

theprathamjain@gmail.com

Abstract

Traditional search systems such as BM25 rely on exact term matching, which works well when the query closely overlaps with the document text. In many practical cases, small changes in phrasing or vocabulary are enough to hide relevant results. To handle this, I experimented with combining lexical retrieval and dense semantic embeddings in a single pipeline.

The system stores document embeddings in an int8 format to reduce memory usage, while still computing semantic similarity using cosine distance. Final ranking is done using a weighted combination of BM25 and dense scores.

I evaluated the setup only on a subset of the MS MARCO passage ranking dataset, focusing only on queries with a single labeled relevant passage to keep the comparison controlled. Across both NDCG@10 and MRR@10, the hybrid approach consistently performed better than a purely lexical baseline. At the same time, quantizing embeddings reduced storage by roughly 75% without causing any noticeable drop in retrieval quality.

The results suggested that adding a lightweight semantic layer on top of BM25 can improve ranking performance without introducing significant computational or storage overhead.

1 Introduction

BM25 remains widely used in search systems because it is simply efficient, and scales well. But its reliance on exact token overlap can become a limitation when queries and documents use different wording for the same idea. In practice, this shows up as missed results even when relevant documents exist.

Dense retrieval models based on transformer encoders address this by mapping text into vector representations that capture meaning rather than just tokens. The downside is that these embeddings are expensive to store and compute, especially at scale.

In this work, I explored a hybrid setup that combines both approaches. The goal was to see how much improvement we could get by adding semantic signals on top of BM25 while keeping resource usage under control. To make this practical, we store embeddings in an int8 format instead of float32.

The main things I focused on were:

- how much dense retrieval improves ranking over BM25,
- whether quantization affects retrieval quality,
- how sensitive the hybrid score is to interpolation weights,
- and what the actual storage and latency costs look like.

2 Background

2.1 Lexical Retrieval

BM25 scores a document d with respect to a query q based on term frequency and idf:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d)(k_1 + 1)}{tf(t, d) + k_1(1 - b + b \frac{|d|}{\text{avgdl}})}$$

2.2 Dense Retrieval

Dense retrieval represents both queries and documents as vectors and compares them using cosine similarity:

$$S_{\text{dense}}(q, d) = \cos(\mathbf{e}_q, \mathbf{e}_d)$$

I tried a few encoder models initially. But Larger models did give slightly better scores in early tests, but the added latency and memory usage did not justify this for me. For this reason, I used *multi-qa-MiniLM-L6-cos-v1*, which was a good balance between performance and efficiency.

2.3 Hybrid Scoring

The final score combines lexical and semantic components:

$$S_{\text{hybrid}} = \alpha S_{\text{dense}} + (1 - \alpha) S_{\text{lexical}}$$

3 Experimental Setup

3.1 Dataset and Metrics

All experiments were conducted on the MS MARCO passage ranking dataset. To keep the evaluation straightforward, I filtered out queries with multiple relevant passages and only kept those with exactly one labeled answer. This resulted in a small set of just 696 queries.

Performance was measured using NDCG@10 and MRR@10. Statistical significance was checked using paired t-tests and Wilcoxon signed rank tests.

4 Results

4.1 Retrieval Effectiveness

Method	NDCG@10	MRR@10
BM25	0.5674	0.4330
Dense (Float32)	0.6691	0.5629
Dense (Int8)	0.6678	0.5612

Table 1: Comparison of lexical and dense retrieval performance.

Dense retrieval improved ranking performance over BM25 across both metrics. The difference was statistically significant ($p < 10^{-15}$). When comparing float32 and int8 embeddings, the difference in performance was very small and not statistically significant ($p > 0.4$).

4.2 Latency and Storage

Metric	Float32	Int8
Embedding Size (MB)	8.37	2.09
Compression Ratio	4x	
Space Saved	75%	
Full Pipeline Latency (ms)	7.73	

Table 2: Storage and latency comparison across precision formats.

Switching to int8 reduced the storage requirements a lot while keeping latency almost unchanged in the setup.

5 Ablation Study

5.1 Alpha Sensitivity

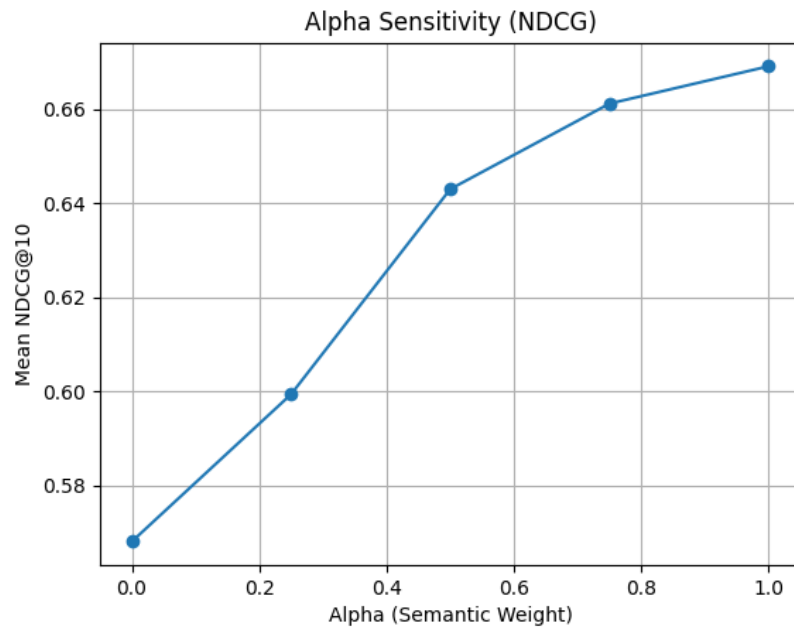


Figure 1: Effect of interpolation weight on NDCG@10.

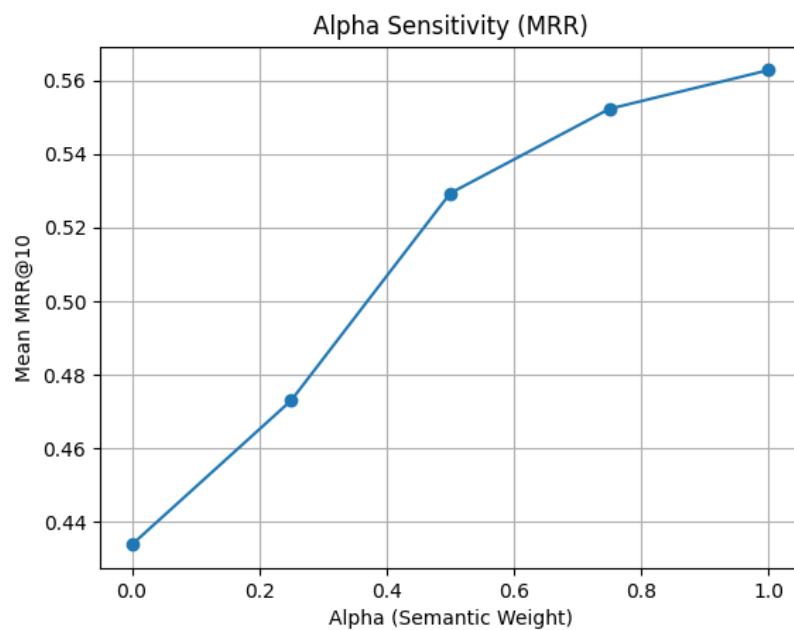


Figure 2: Effect of interpolation weight on MRR@10.

As the weight on the dense component increased, performance generally improved. But beyond a certain point, the gains started to level off.

5.2 Quantization Comparison

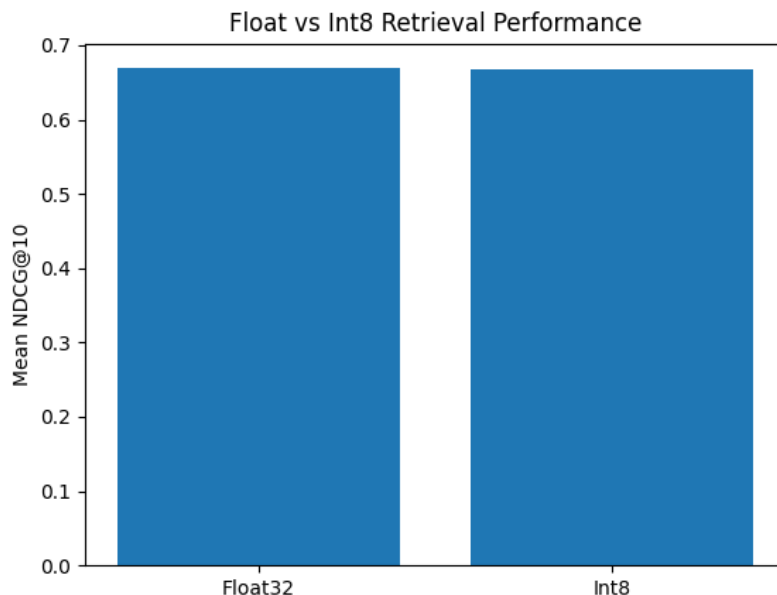


Figure 3: Performance comparison between float32 and int8 embeddings.

The gap between float32 and int8 was actually minimal across both metrics, meaning that quantization did not meaningfully degrade retrieval quality in this setup.

6 Discussion

The results indicate that semantic embeddings help capture relationships that BM25 alone misses, especially when queries and documents differ in wording. This also meant that the hybrid setup avoids relying entirely on dense retrieval, so the system is efficient.

One limitation I noticed was that the current evaluation has the use of single-relevance queries. It is so because this makes only comparison easier, but then it does not fully reflect real-world search scenarios where multiple documents may be relevant. Extending the evaluation to such cases would be a useful next step.

7 Conclusion

I implemented a hybrid retrieval system that combines BM25 with quantized dense embeddings. The approach improves ranking performance while keeping storage and latency costs manageable.

From a practical standpoint, this suggests that adding a semantic layer on top of existing lexical systems can be a reasonable upgrade without requiring major infrastructure changes.

References
